Proceso:	Guía de implementación de Botón de pagos - React
Clasificación de la información:	☑ Pública ☐ Interna ☐ Confidencial
Versión	1.2
Custodio:	Aseguramiento de la calidad (QA)
Código	

Guía de implementación de botón de pagos React

Febrero de 2026

Av. Eloy Alfaro 44 -110 y Río Coca +593 96 428 2244

HISTORIAL DE REVISIONES

Las versiones cambiarán únicamente cuando este documento ya haya sido firmado por el personal de aprobación, de lo contrario únicamente se debe registrar la actividad realizada sobre el documento manteniendo la misma versión.

Fecha	Versión	Descripción	Autor
13-04-2023	1.0	Creación del Documento.	Javier Vallejo
01-04-2025	1.1	Actualización de Formato	Cristian Moreira
26-02-2026	1.2	Mejora de control de Pago Recurrente	Guillermo Guachamín

Av. Eloy Alfaro 44 -110 y Río Coca
+593 96 428 2244

Plux

Tabla de contenido

1. Implementación de botón de pagos **4**

1.1 Instalación en el sitio web del comercio 4

1.2 Tarjetas de Prueba 12

Av. Eloy Alfaro 44 -110 y Río Coca +593 96 428 2244

1. Implementación de botón de pagos

Para realizar la implementación del botón de pagos se necesita realizar las siguientes actividades, registro de comercio e instalación del botón.

1.1 Instalación en el sitio web del comercio

- **Llamado a dependencias necesarias para que el API pueda funcionar.**

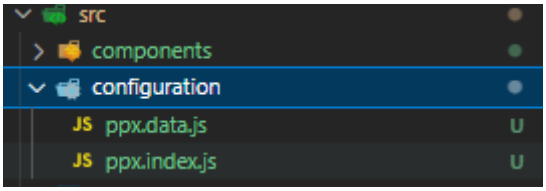
```
<script src="https://code.jquery.com/jquery-3.4.1.min.js"></script>

Sandbox:
<script
src="https://sandbox-paybox.pagoplux.com/paybox/index_angular.js"><
/script>
Producción:
<script
src="https://paybox.pagoplux.com/paybox/index_angular.js"></script>
```

Estos archivos son 100% necesarios y deben ser llamados en el <head> del sitio en el archivo index.html de React, en caso de que el sitio ya cuente con una versión de Jquery no será necesario volver a llamarlo al inicio.

- **Declaración de variables.**

En la ubicación src creamos un nuevo directorio denominado **configuration** con los siguientes archivos **ppx.data.js** y **ppx.index.js.**



ppx.data.js:
Se debe crear la variable **data** con los valores que deben ser rellenados previamente a ser procesado el pago en conjunto con el método **onAuthorize.**

Av. Eloy Alfaro 44 -110 y Río Coca
+593 96 428 2244

```
let data = {
  /* Requerido. Email de la cuenta PagoPlux del Establecimiento */
  PayboxRemail: "string",

  /* Requerido. Email del usuario que realiza el pago */
  PayboxSendmail: "string",

  /* Requerido. Nombre del establecimiento en PagoPlux */
  PayboxRename: "string",

  /* Requerido. Nombre del usuario que realiza el pago */
  PayboxSendname: "string",

  /* Requerido. Monto total de productos o servicios que no aplican
impuestos, máximo 2 decimales.
     Ejemplo: 100.00, 10.00, 1.00 */
  PayboxBase0: "string",

  /* Requerido. Monto total de productos o servicios que aplican
impuestos, el valor debe incluir el impuesto, máximo 2 decimales.
     Ejemplo: 100.00, 10.00, 1.00 */
  PayboxBase12: "string",

  /* Requerido. Descripción del pago */
  PayboxDescription: "string",

  /* Requerido Tipo de Ejecución
   * Production: true  (Modo Producción)
   * Production: false (Modo Prueba)
   */
  PayboxProduction: false,

  /* Requerido Ambiente de ejecución
```


Av. Eloy Alfaro 44 -110 y Río Coca +593 96 428 2244

```
   * prod:    Modo Producción
   * sandbox: Modo Prueba
   */
  PayboxEnvironment: "sandbox",

  /* Requerido. Lenguaje del Paybox
   * Español: es
   */
  PayboxLanguage: "es",

  /* Requerido. Identifica el tipo de iframe de pagoplux por defecto
true */
  PayboxPagoPlux: true,

  /* Requerido. Dirección del tarjetahabiente */
  PayboxDirection: "string",

  /* Requerido. Teléfono del tarjetahabiente */
  PayBoxClientPhone: "número telefónico del tarjetahabiente",

  /* Requerido. Identificación del tarjetahabiente */
  PayBoxClientIdentification: "string",

  /* SOLO PARA PAGOS RECURRENTES (solo aplica para comercios con pagos
recurrentes habilitados) */
  /* Requerido
   * true  -> en caso de realizar un pago recurrente
   * false -> si es pago normal
   */
  PayboxRecurrent: true,

  /* Requerido. Id o nombre exacto del plan registrado en el comercio
en la plataforma de pagoplux */
```


Av. Eloy Alfaro 44 -110 y Río Coca +593 96 428 2244

```
  PayboxIdPlan: "Plan Mensual",

  /* true  -> cobros automáticos según la frecuencia del plan
   * false -> cobros mediante solicitud
   */
  PayboxPermitirCalendarizar: true,

  /* Requerido
   * true  -> El débito se realiza en el momento del pago
   * false -> El débito se realizará en la fecha de corte según el plan
contratado
   */
  PayboxPagoInmediato: false,

  /* Requerido
   * true  -> realiza un cobro de prueba de 1$ y reverso automático
   * false -> no se realizará ningún cobro de prueba
   * NOTA: PayboxPagoInmediato debe ser igual a false
   */
  PayboxCobroPrueba: false,
  onAuthorize: (response) => {
    if (response.status === "succeeded") {
      console.log(response);
      alert("Proceso completado con éxito");
    }
  },
};
```

La variable PayboxRemail debe contener el correo electrónico del comercio registrado en "**PAGOPLUX**" ya que se valida durante el proceso de pago la existencia del registro e identificar los datos MID y TID asignados.

ppx.index.js:
Se debe crear las siguientes funciones para el llamado, envío de datos e inicialización del componente.
En la ubicación src/assets/js/ se debe crear un archivo JS con el nombre **indexPagoPlux.js** con el siguiente código.

```
const iniciarDatos=(dataPago)=> {
    if (Data) {
        Data.init(dataPago);
    }
  }
const reload=(data)=> {
    if (Data) {
      Data.reload(data);
    }
  }
export{iniciarDatos,reload};
```

- **Botón.**

Creamos un nuevo componente react denominado PpxButton, en el cual debemos importar nuestro ppx.index.js y llamar la función iniciarDatos dentro del hook useEffect como se detalla a continuación:

```
import { useEffect } from "react";
import { iniciarDatos } from "../configuration/ppx.index";

const PpxButton = ({ data }) => {
  const estiloBoton = {
    display: "none",
    backgroundColor: "#FAFAFA",
    right: "80px",
```


Av. Eloy Alfaro 44 -110 y Río Coca
+593 96 428 2244

```
    backgroundImage:
      "url(https://sandbox-paybox.pagoplux.com/img/pagar.png?v1)",
    backgroundRepeat: "no-repeat",
    backgroundPosition: "center",
    height: "96px",
    width: "215px",
    border: "none",
    cursor: "pointer",
    backgroundSize: "contain",
    outline: "0",
    boxShadow: "0px 2px 2px lightgray",
  };
  useEffect(() => {
    iniciarDatos(data);
  }, [data]);
  return (
    <>
      <div id="modalPaybox"></div>
      <button style={estiloBoton} id="pay" type="submit"></button>
    </>
  );
};
export default PpxButton;
```

Ahora podemos llamar a nuestro botón de pagos desde otros componentes, es necesario enviar la variable data como props del componente PpxButton.
Se debe presentar el botón pagar ahora:

```
function App() {
  return (
    <div className="App">
```

```
      <ResponsiveAppBar />
      <header className="App-header">
        <PpxButton data={data} />
      </header>
    </div>
  );
}
export default App;
```

Se debe presentar el botón de la siguiente:

Figura 1: ***Botón de pagos***

Al hacer click en el botón se desplegará el Paybox el cual utiliza la información suministrada mediante JSON para presentar los campos de Paybox.

Nota: Los estilos varían según las necesidades del comercio.

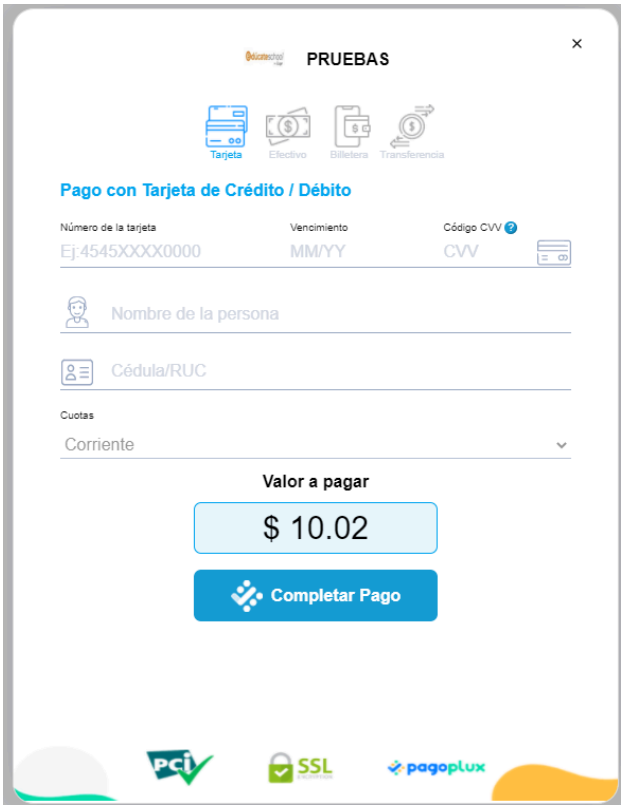


*Figura 2: **Ventana Popup Paybox***

- **Respuesta**.

Método que se ejecuta como resultado del proceso de pago, cuando la transacción se ha realizado de manera correcta, se ejecutará el método onAuthorize y la respuesta "response" contendrá la información del resultado del pago. Si response.status es igual a "succeded" el pago se ha realizado correctamente y se puede utilizar la información para el mensaje en el comercio o registró en el comercio. En caso de cerrar el popup o cancelar el proceso de compra no se ejecuta el método.

```
// ejemplo metodo onAuthorize
onAuthorize: (response) => {
    if (response.status === 'succeeded') {
      jQuery('.container-unpayed').hide();
      response.amount, //monto
```


Av. Eloy Alfaro 44 -110 y Río Coca
+593 96 428 2244

```
        response.deferred, //diferidos
        response.interest, //tiene intereses
        response.interestValue, //monto intereses
        response.amountWoTaxes, //monto impuestos
        response.cardInfo, //número de tarjeta encriptado
        response.cardIssuer, //marca tarjeta Ejemplo: Visa
        response.cardType, //tipo tarjeta Ejemplo: Crédito
        response.clientID, //identificación tarjetahabiente
        response.clientName, //nombre tarjetahabiente
        response.fecha, //fecha de pago
        response.id_transaccion, //id de transacción pagoplux
        response.state, //estado del pago
        response.token, //voucher del pago
        response.tipoPago //tipo de pago usado
    }
}
```

Una vez el pago sea efectuado correctamente, en esta sección es donde deberá ir el proceso/función a ejecutarse una vez procesado el pago. Mediante Javascript se puede invocar a procesos del comercio para completar el proceso de pago en el comercio.

1.2 Tarjetas de Prueba

Para realizar pruebas en "Modo Sandbox" cuando se esté usando el SandBox del API, aquí se encuentra el listado de tarjetas que se procesarán exitosamente.

Esta tarjeta no es válida en "Producción".

Marca	Nombres	Número	Expiración	CVV	OTP	Respuesta
VISA	John doe	4540639936908783	abr-29	123		Éxito - Pago exitoso

Av. Eloy Alfaro 44 -110 y Río Coca
+593 96 428 2244

AMERICAN EXPRESS	John doe	376651861404404	abr-29	1234		Éxito - Pago exitoso
MASTERCARD	John doe	5230428590692129	abr-29	123		Éxito - Pago exitoso
DINERS	John doe	36417200103608	abr-29	123	123456	Éxito - Pago exitoso
DISCOVER	John doe	6011761603370843	abr-29	123	123456	Éxito - Pago exitoso
VISA	John doe	4540633357674263	abr-29	123		Error - Cupo no disponible
AMERICAN EXPRESS	John doe	376651681114324	abr-29	1234		Error - Cupo no disponible
MASTERCARD	John doe	5230426513413979	abr-29	123		Error - Cupo no disponible
DINERS	John doe	36707777533427	abr-29	123	123456	Error - Cupo no disponible
DISCOVER	John doe	6011635662534327	abr-29	123	123456	Error - Cupo no disponible
VISA	John doe	4540636579673146	abr-29	123		Error - Tarjeta Expirada
AMERICAN EXPRESS	John doe	376651373069836	abr-29	1234		Error - Tarjeta Expirada
MASTERCARD	John doe	5230422993522090	abr-29	123		Error - Tarjeta Expirada
DINERS	John doe	36899627890729	abr-29	123	123456	Error - Tarjeta Expirada
DISCOVER	John doe	6011584522711685	abr-29	123	123456	Error - Tarjeta Expirada
VISA	John doe	4540632093612050	abr-29	123		Error - Tarjeta Bloqueada
AMERICAN EXPRESS	John doe	376651198045250	abr-29	1234		Error - Tarjeta Bloqueada
MASTERCARD	John doe	5230424748383364	abr-29	123		Error - Tarjeta Bloqueada
DINERS	John doe	30513704308648	abr-29	123	123456	Error - Tarjeta Bloqueada
DISCOVER	John doe	6011550021804842	abr-29	123	123456	Error - Tarjeta Bloqueada
VISA	John doe	4540639405966494	abr-29	123		Ok - Día Par, Error - Día Impar
AMERICAN EXPRESS	John doe	376651368448755	abr-29	1234		Ok - Día Par, Error - Día Impar
MASTERCARD	John doe	5230424845336083	abr-29	123		Ok - Día Par, Error - Día Impar
DINERS	John doe	38164167149624	abr-29	123	123456	Ok - Día Par, Error - Día Impar
DISCOVER	John doe	6011994276172051	abr-29	123	123456	Ok - Día Par, Error - Día Impar

Nota: Para realizar las pruebas se debe usar una cédula ecuatoriana válida.